

# Softwarekonzept V1

ClimateCanary – Raumklima Analyse für Firmen

PS Software Engineering SS2026

Maria Kuhn      Fabienne Schedler      Emma Danko      Prahbdip Singh  
Adriano Paganini

Deadline: 12.03.2026

## Inhaltsverzeichnis

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Systemüberblick</b>                                    | <b>3</b>  |
| 1.1      | Zielsetzung . . . . .                                     | 3         |
| 1.2      | Zielgruppe . . . . .                                      | 4         |
| 1.3      | Benutzer und Rollen . . . . .                             | 4         |
| 1.4      | Kernfunktionalitäten . . . . .                            | 4         |
| <b>2</b> | <b>Use Case Diagramm</b>                                  | <b>5</b>  |
| 2.1      | Akteure . . . . .                                         | 5         |
| 2.2      | Übersicht der Use Cases . . . . .                         | 6         |
| <b>3</b> | <b>Use Case Beschreibungen</b>                            | <b>6</b>  |
| 3.1      | UC-01: Raumklimadaten eigenes Büro einsehen . . . . .     | 6         |
| 3.2      | UC-02: Raumklimadaten Allgemeinflächen einsehen . . . . . | 7         |
| 3.3      | UC-03: Abteilungsübersicht einsehen . . . . .             | 7         |
| 3.4      | UC-04: Grenzwertwarnung – Auslösung und Anzeige . . . . . | 7         |
| 3.5      | UC-05: Grenzwerte konfigurieren . . . . .                 | 8         |
| 3.6      | UC-06: Abwesenheit eintragen . . . . .                    | 8         |
| 3.7      | UC-07: Geräte anlegen und konfigurieren . . . . .         | 9         |
| <b>4</b> | <b>Klassendiagramm V1</b>                                 | <b>9</b>  |
| 4.1      | Klassenbeschreibungen . . . . .                           | 10        |
| 4.2      | Wichtige Assoziationen . . . . .                          | 10        |
| 4.3      | Löschen von Daten . . . . .                               | 11        |
| <b>5</b> | <b>GUI Prototyp</b>                                       | <b>11</b> |
| 5.1      | Login . . . . .                                           | 11        |
| 5.2      | Dashboard (Mitarbeiter:in) . . . . .                      | 11        |
| 5.3      | Verlaufsansicht (Raumdetails) . . . . .                   | 12        |
| 5.4      | Dashboard (Abteilungsleitung) . . . . .                   | 12        |
| 5.5      | Dashboard (Höheres Management) . . . . .                  | 12        |
| 5.6      | Konfigurationsansicht (Gebäudeadmin) . . . . .            | 12        |

|          |                                |           |
|----------|--------------------------------|-----------|
| 5.7      | Gerätemanagement (Systemadmin) | 12        |
| <b>6</b> | <b>Projektplan</b>             | <b>13</b> |
| 6.1      | Verantwortlichkeiten           | 13        |
| 6.2      | Meilensteine und Zeitplan      | 14        |
| 6.3      | Inkrementelle Entwicklung      | 14        |
| 6.4      | Zusätzliche Aufgaben           | 15        |

# 1 Systemüberblick

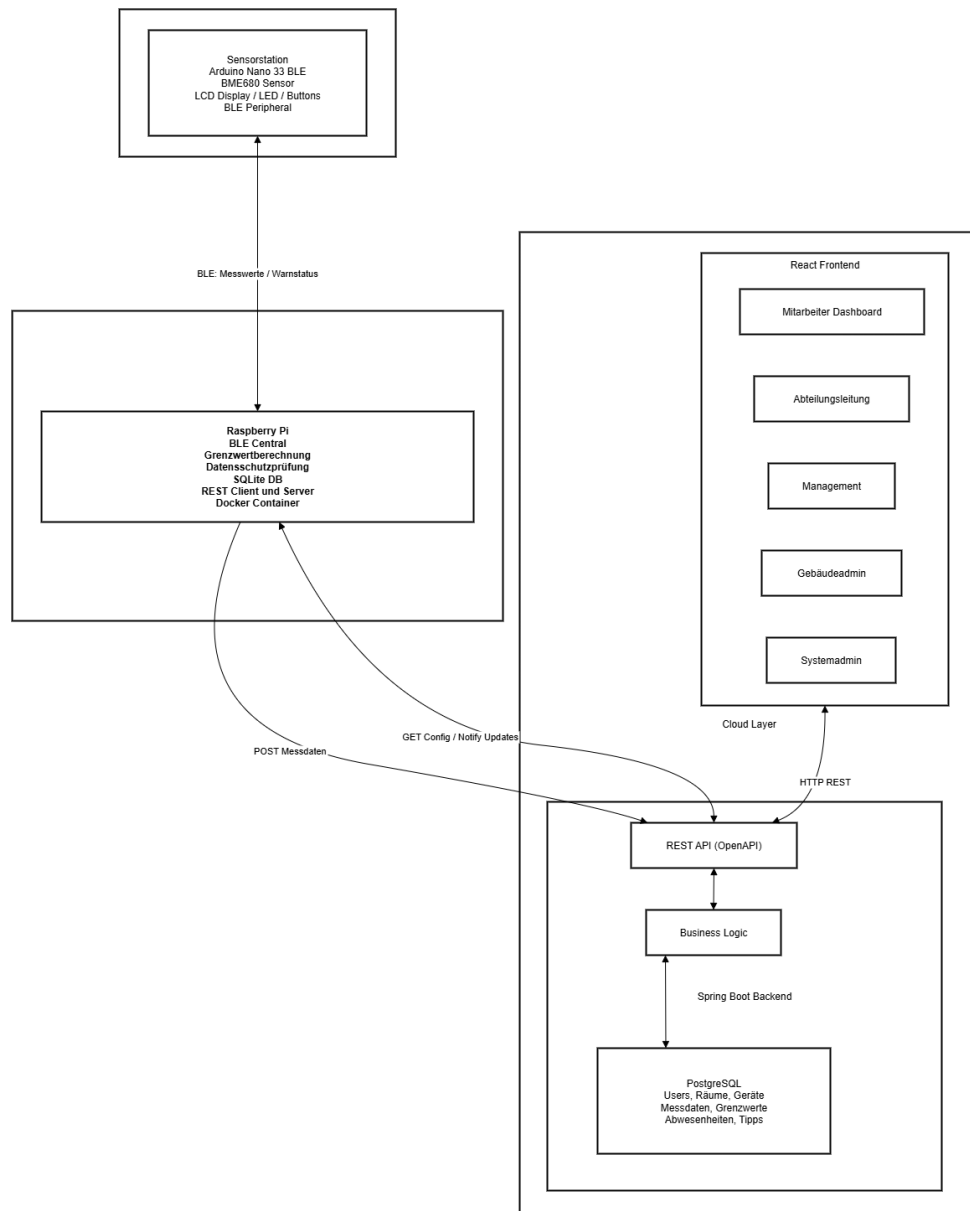


Abbildung 1: System architecture draft

## 1.1 Zielsetzung

- ClimateCanary ist ein IoT-basiertes Monitoring-System, das Unternehmen eine kontinuierliche und automatisierte Überwachung des Raumklimas in Bürogebäuden ermöglicht.
- Das System erfasst Temperatur, Luftfeuchtigkeit und Luftqualität in Echtzeit und stellt diese Daten rollenabhängig in einer zentralen Webanwendung dar.
- Kurzfristiges Ziel: Mitarbeiter:innen und Verwaltung sollen jederzeit informiert sein, wenn Raumklimawerte kritische Grenzwerte über- oder unterschreiten, und können entsprechend reagieren.

- Mittelfristiges Ziel: Langfristige Auswertungen der gesammelten Daten sollen als Entscheidungsgrundlage für gezielte Maßnahmen zur Raumklimaverbesserung dienen (z.B. Anschaffung von Klimaanlage, Luftreinigern).
- Langfristiges Ziel (außerhalb des aktuellen Projektumfangs): Automatische Steuerung von Heizung, Jalousien und Fenstern auf Basis der gesammelten Klimadaten.

## 1.2 Zielgruppe

- Primäre Zielgruppe sind mittelständische bis große Unternehmen mit mehreren Büroräumen und Allgemeinflächen.
- Das System richtet sich an alle Mitarbeiter:innen eines Unternehmens, die das Raumklima an ihrem Arbeitsplatz überwachen möchten.
- Zusätzlich adressiert es Abteilungsleitungen, das höhere Management sowie die Gebäude- und Systemadministration, die das System konfigurieren und übergreifend auswerten.

## 1.3 Benutzer und Rollen

- **Mitarbeiter:in:** Überwacht das Raumklima am eigenen Arbeitsplatz und in Allgemeinflächen der eigenen Abteilung. Verwaltet eigene Abwesenheiten.
- **Abteilungsleitung:** Erhält eine Übersicht über alle Räume der eigenen Abteilung in reduzierter Datengranularität. Sieht Abwesenheiten der Mitarbeiter:innen.
- **Höheres Management:** Sieht firmenweite, aggregierte und anonymisierte Übersichten und Trendindikatoren. Kein Zugriff auf raumgenaue Daten.
- **Gebäudeadministration:** Sieht alle Raumklimadaten aller Räume, konfiguriert Grenzwerte und verwaltet Raumklima-Tipps. Kein Zugriff auf Userdaten.
- **Systemadministration:** Verwaltet Benutzer, Gebäudestruktur und Geräte (Raspberry Pi, Sensorstationen). Kein Zugriff auf Messdaten.

## 1.4 Kernfunktionalitäten

- Automatische Erfassung von Temperatur, Luftfeuchtigkeit und Luftqualität durch Arduino-Sensorstationen.
- Übertragung der Messdaten via Bluetooth Low Energy (BLE) an einen raumgebundenen Raspberry Pi.
- Lokal am Raspberry Pi: Grenzwertprüfung, datenschutzkonforme Zwischenspeicherung, Weiterleitung an den zentralen Webserver.
- Visuelle Rückmeldung direkt an der Sensorstation: LED-Ampel und LCD-Display mit aktuellen Werten, Warnungen und Tipps.
- Zentrale Webanwendung mit rollenabhängigen Ansichten, Verlaufsdiagrammen, Grenzwertwarnungen und Konfigurationsoptionen.
- Datenschutzkonforme Datenhaltung: Bürodaten werden nur bei Mindestbelegung von 5 Personen persistent gespeichert.

- Abwesenheitssystem zur Pflege von Urlaub, Krankheit etc., das automatisch die Datenschutzprüfung beeinflusst.

## 2 Use Case Diagramm

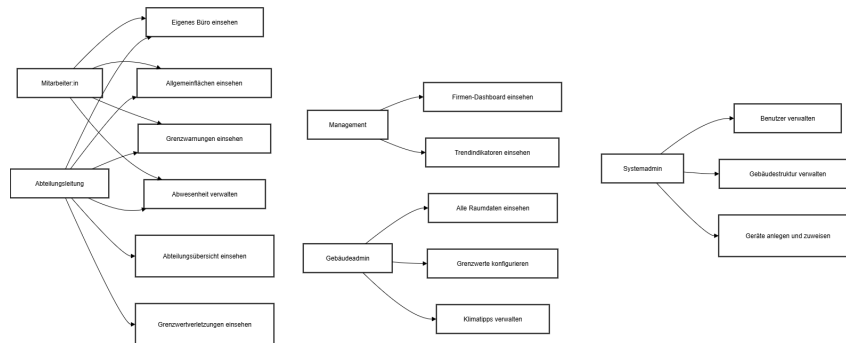


Abbildung 2: Uses cases draft

Das Use Case Diagramm befindet sich als separates Mermaid-Diagramm im GitLab Wiki (Datei: `usecase_diagram.mermaid`).

### 2.1 Akteure

- **Mitarbeiter:in** – Basisrolle; hat Zugriff auf eigenes Büro und Allgemeinflächen der Abteilung.
- **Abteilungsleitung** – Erbt Rechte der Mitarbeiter:in; zusätzlich Abteilungsübersicht.
- **Höheres Management** – Nur firmenweite, aggregierte Sicht; keine Raumdetails.
- **Gebäudeadministration** – Vollzugriff auf Raumklimadaten und Konfiguration; kein Zugriff auf Userdaten.
- **Systemadministration** – Vollzugriff auf Benutzer- und Geräteverwaltung; kein Zugriff auf Messdaten.
- **Raspberry Pi** – Technischer Akteur; übermittelt Messdaten und empfängt Konfigurationsupdates.
- **Sensorstation (Arduino)** – Technischer Akteur; misst und zeigt Daten an; empfängt Warnstatus.

## 2.2 Übersicht der Use Cases

| Akteur            | Use Cases                                                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Mitarbeiter:in    | Raumklima eigenes Büro einsehen (wenn Belegung $\geq 5$ ), Allgemeinflächen einsehen, Grenzwarnungen einsehen, Abwesenheit eintragen/verwalten  |
| Abteilungsleitung | + Abteilungsübersicht einsehen, Büros (Tagesdurchschnitte) einsehen, Abwesenheiten Abteilung einsehen, Grenzwertverletzungen Abteilung einsehen |
| Management        | Firmen-Dashboard einsehen, Trendindikatoren einsehen, anonymisierte Verletzungsstatistiken einsehen                                             |
| Gebäudeadmin      | Alle Raumdaten einsehen, Grenzwerte konfigurieren, Raumklima-Tipps verwalten                                                                    |
| Systemadmin       | Benutzer verwalten, Gebäudestruktur verwalten, Geräte anlegen und zuweisen                                                                      |

## 3 Use Case Beschreibungen

### 3.1 UC-01: Raumklimadaten eigenes Büro einsehen

- **Akteur:** Mitarbeiter:in
- **Vorbedingung:** Nutzer:in ist eingeloggt und einem Büroraum zugeordnet. Mindestens 5 Personen des Raums sind aktuell anwesend (keine Abwesenheit erfasst).
- **Normalablauf:**
  1. Nutzer:in öffnet die Raumsicht des eigenen Büros.
  2. System prüft die aktuelle Belegung des Raums.
  3. System zeigt aktuelle Messwerte (Temperatur, Luftfeuchtigkeit, Luftqualität) und Verlaufsdiagramme (Tages-/Wochen-/Monatsansicht) an.
  4. Konfigurierte Grenzwerte sind als Referenzlinien in den Diagrammen sichtbar.
  5. Aktive Grenzwertwarnungen werden prominent angezeigt, inkl. Raumklima-Tipps.
- **Alternativablauf – Mindestbelegung nicht erfüllt:**
  1. System erkennt, dass die Belegung unter 5 Personen liegt.
  2. System zeigt Hinweis: “Keine Daten verfügbar – Datenschutzbedingungen nicht erfüllt.”
  3. Keine Messdaten werden angezeigt.
- **Alternativablauf – Verbindungsausfall:**
  1. Für einen bestimmten Zeitraum fehlen Messdaten (z.B. Raspberry Pi offline).
  2. System markiert Lücken in den Verlaufsdiagrammen deutlich (z.B. grau hinterlegte Bereiche).

### 3.2 UC-02: Raumklimadaten Allgemeinflächen einsehen

- **Akteur:** Mitarbeiter:in, Abteilungsleitung
- **Vorbedingung:** Nutzer:in ist eingeloggt und einer Abteilung zugeordnet.
- **Normalablauf:**
  1. Nutzer:in wählt eine Allgemeinfläche (Konferenzraum, Aufenthaltsraum etc.) der eigenen Abteilung aus.
  2. System zeigt aktuelle Messwerte und Verlaufsdiagramme in voller Granularität an.
  3. Keine Datenschutzeinschränkungen – Daten sind immer verfügbar.

### 3.3 UC-03: Abteilungsübersicht einsehen

- **Akteur:** Abteilungsleitung
- **Vorbedingung:** Nutzer:in ist als Abteilungsleitung eingeloggt.
- **Normalablauf:**
  1. Abteilungsleitung öffnet die Abteilungsübersicht.
  2. System zeigt eine vergleichende Darstellung aller Räume der Abteilung (aktuelle Messwerte, Anzahl aktiver Warnungen).
  3. Für Büros oberhalb der Mindestbelegung: Verlaufsansichten in reduzierter Granularität (Tagesdurchschnitte).
  4. Für Allgemeinflächen: Verlaufsansichten in voller Granularität.
  5. Aktive und vergangene Grenzwertverletzungen auf Raumbene sind einsehbar.

### 3.4 UC-04: Grenzwertwarnung – Auslösung und Anzeige

- **Akteur:** Raspberry Pi (technisch), Mitarbeiter:in / Abteilungsleitung (sichtbar)
- **Vorbedingung:** Grenzwerte sind für den betroffenen Raum konfiguriert. Sensorstation ist aktiv und verbunden.
- **Normalablauf:**
  1. Raspberry Pi empfängt Messdaten von der Sensorstation.
  2. Raspberry Pi prüft Messwerte gegen konfigurierte Grenzwerte mit Noise-Filter (kein Auslösen bei kurzfristigen Ausreißern).
  3. Bei bestätigter Grenzwertüberschreitung: Raspberry Pi setzt Warnstatus und sendet diesen an die Sensorstation und die Webapp.
  4. Sensorstation: LED wechselt auf Rot, Display zeigt Warnmodus mit Hinweis und Tipp.
  5. Webapp: Warnung wird prominent für alle Nutzer:innen des betroffenen Raums angezeigt, inkl. konfiguriertem Raumklima-Tipp.

6. Abteilungsleitung sieht Warnung in der Abteilungsübersicht.

- **Alternativablauf – Warnung aufheben:**

1. Messwerte kehren dauerhaft in den Normalbereich zurück.
2. Raspberry Pi hebt Warnstatus auf und informiert Sensorstation und Webapp.
3. LED wechselt auf Grün, Warnmeldung in der Webapp verschwindet. Vergangene Verletzung bleibt historisch gespeichert.

- **Designentscheidung:** Eine Grenzwertwarnung wird erst ausgelöst, wenn ein Grenzwert über einen konfigurierbaren Zeitraum (z.B. 3 aufeinanderfolgende Messungen) überschritten wird. Dadurch werden kurzfristige Ereignisse (Fenster kurz öffnen, Atemstoß auf Sensor) gefiltert.

### 3.5 UC-05: Grenzwerte konfigurieren

- **Akteur:** Gebäudeadministration

- **Vorbedingung:** Nutzer:in ist als Gebäudeadmin eingeloggt. Raum und Sensorstation sind im System angelegt.

- **Normalablauf:**

1. Gebäudeadmin öffnet die Konfigurationsansicht eines Raums.
2. Admin setzt oder ändert obere Grenzwerte für Temperatur, Luftfeuchtigkeit und Luftqualität. Untere Grenzwerte sind optional.
3. System speichert die neuen Grenzwerte in der PostgreSQL-Datenbank.
4. System informiert den zuständigen Raspberry Pi über die Änderung per REST-Benachrichtigung.
5. Raspberry Pi ruft die aktualisierten Grenzwerte ab und speichert sie lokal in der SQLite-Datenbank.

### 3.6 UC-06: Abwesenheit eintragen

- **Akteur:** Mitarbeiter:in

- **Vorbedingung:** Nutzer:in ist eingeloggt und einem Büroraum zugeordnet.

- **Normalablauf:**

1. Mitarbeiter:in trägt eine Abwesenheit (Urlaub, Krankheit, etc.) stunden- oder tageweise ein.
2. System prüft, ob durch diese Abwesenheit die Mindestbelegung (5 Personen) im Büroraum unterschritten wird.
3. Wenn ja: System informiert den zuständigen Raspberry Pi. Ab diesem Zeitpunkt werden keine Messdaten für diesen Raum persistent gespeichert oder übertragen.
4. Wenn nein: Keine Änderung am Datenspeicherverhalten.



### 3.7 UC-07: Geräte anlegen und konfigurieren

- **Akteur:** Systemadministration
- **Vorbedingung:** Nutzer:in ist als Systemadmin eingeloggt. Gebäudestruktur (Gebäude, Abteilungen, Räume) ist bereits angelegt.
- **Normalablauf:**
  1. Systemadmin legt ein neues Raspberry Pi-Objekt an. System generiert automatisch eine eindeutige ID.
  2. Systemadmin ordnet den Raspberry Pi einem Raum zu.
  3. Systemadmin legt eine neue Sensorstation an. System generiert automatisch eine eindeutige ID.
  4. Systemadmin ordnet die Sensorstation dem Raspberry Pi zu.
  5. Die generierten IDs werden für die physische Konfiguration der Geräte verwendet (hardcoded auf Arduino, conf.yaml auf Raspberry Pi).

## 4 Klassendiagramm V1

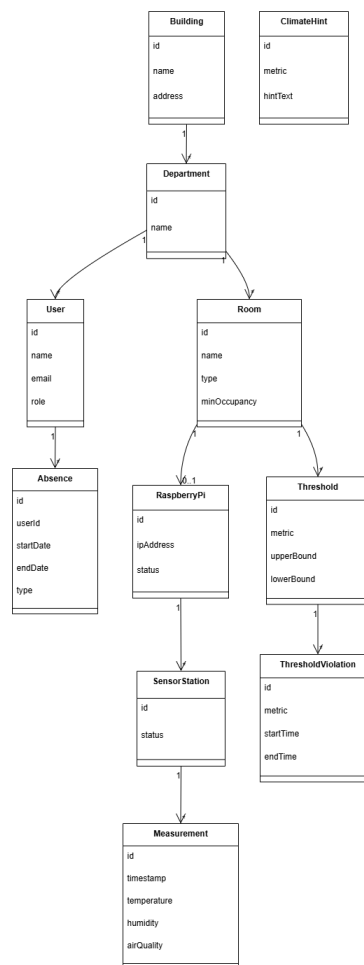


Abbildung 3: Class diagram draft

*Das vollständige Klassendiagramm befindet sich als separates Mermaid-Diagramm im Git-Lab Wiki (Datei: `class_diagram_v1.mermaid`).*

## 4.1 Klassenbeschreibungen

- **User** – Repräsentiert einen Systembenutzer. Attribute: id, name, email, password-Hash, role. Assoziationen: gehört einer Abteilung an, ist einem Büroraum zugeordnet, hat Abwesenheiten.
- **Role** – Enum: EMPLOYEE, DEPARTMENT\_LEAD, MANAGEMENT, BUILDING\_ADMIN, SYSTEM\_ADMIN.
- **Absence** – Repräsentiert eine Abwesenheit eines Users. Attribute: id, userId, startDate, endDate, type (URLAUB, KRANKHEIT, KARENZ, etc.).
- **Building** – Repräsentiert ein Gebäude. Attribute: id, name, address. Enthält Abteilungen.
- **Department** – Repräsentiert eine Abteilung. Attribute: id, name, buildingId. Enthält Räume und User.
- **Room** – Repräsentiert einen Raum. Attribute: id, name, type (OFFICE / COMMON\_AREA), departmentId, minOccupancy (default: 5 für Büroräume). Assoziationen: hat einen Raspberry Pi, hat Grenzwerte, hat Messdaten.
- **RaspberryPi** – Repräsentiert einen Raspberry Pi. Attribute: id (generiert), roomId, ipAddress, status. Assoziationen: hat Sensorstationen, hat Grenzwerte (lokal gespiegelt).
- **SensorStation** – Repräsentiert eine Sensorstation (Arduino). Attribute: id (generiert, hardcoded auf Arduino), raspberryPiId, status.
- **Measurement** – Repräsentiert einen Messdatenpunkt. Attribute: id, sensorStationId, timestamp, temperature, humidity, airQuality. Wird nur bei erfüllter Datenschutzbedingung persistent gespeichert.
- **Threshold** – Repräsentiert Grenzwerte für einen Raum. Attribute: id, roomId, metric (TEMPERATURE / HUMIDITY / AIR\_QUALITY), upperBound, lowerBound (optional).
- **ThresholdViolation** – Repräsentiert eine aktive oder historische Grenzwertverletzung. Attribute: id, roomId, thresholdId, startTime, endTime (null wenn noch aktiv), metric, value.
- **ClimateHint** – Repräsentiert einen Raumklima-Tipp. Attribute: id, metric, condition, hintText. Wird bei Grenzwertverletzungen angezeigt.

## 4.2 Wichtige Assoziationen

- **Building 1 – \* Department**: Ein Gebäude hat mehrere Abteilungen.
- **Department 1 – \* Room**: Eine Abteilung hat mehrere Räume.
- **Department 1 – \* User**: Eine Abteilung hat mehrere User.

- **Room 1 – 0..1 RaspberryPi**: Ein Raum hat maximal einen Raspberry Pi.
- **RaspberryPi 1 – \* SensorStation**: Ein Raspberry Pi kann mehrere Sensorstationen verwalten (im Prototyp: eine).
- **SensorStation 1 – \* Measurement**: Eine Sensorstation produziert viele Messdaten.
- **Room 1 – \* Threshold**: Ein Raum hat Grenzwerte pro Messgröße.
- **User 1 – \* Absence**: Ein User kann mehrere Abwesenheiten haben.

### 4.3 Löschen von Daten

- Wird ein User gelöscht, werden seine Abwesenheiten ebenfalls gelöscht (CASCADE).
- Wird eine Sensorstation gelöscht, werden ihre Messdaten ebenfalls gelöscht (CASCADE). Historische Grenzwertverletzungen bleiben erhalten (roomId Referenz bleibt bestehen).
- Wird ein Raum gelöscht, werden alle zugehörigen Messdaten, Grenzwerte und Grenzwertverletzungen gelöscht (CASCADE).
- Wird ein Raspberry Pi aus einem Raum entfernt, bleiben historische Messdaten erhalten; der Raspberry Pi-Eintrag bleibt im System, bis er explizit gelöscht wird.
- Messdaten, die aufgrund der Datenschutzbedingung (Mindestbelegung) nicht gespeichert wurden, existieren nie persistent – sie müssen also nicht gelöscht werden.

## 5 GUI Prototyp

*Der vollständige GUI Prototyp (Mockups) befindet sich als separates Dokument im GitLab Wiki. Hier folgt eine textuelle Beschreibung der Kernansichten.*

### 5.1 Login

- Einfache Login-Maske mit E-Mail und Passwort.
- Nach Login: Weiterleitung auf rollenabhängiges Dashboard.

### 5.2 Dashboard (Mitarbeiter:in)

- Übersichtskacheln: Eigenes Büro (aktuelle Temperatur, Luftfeuchtigkeit, Luftqualität) + Allgemeinflächen der Abteilung.
- Prominente Warnanzeige wenn aktive Grenzwertverletzung im eigenen Büro vorhanden.
- Hinweisbox wenn Bürodaten wegen Datenschutz nicht verfügbar sind.
- Navigation zu: Verlaufsansicht, Abwesenheitsverwaltung.

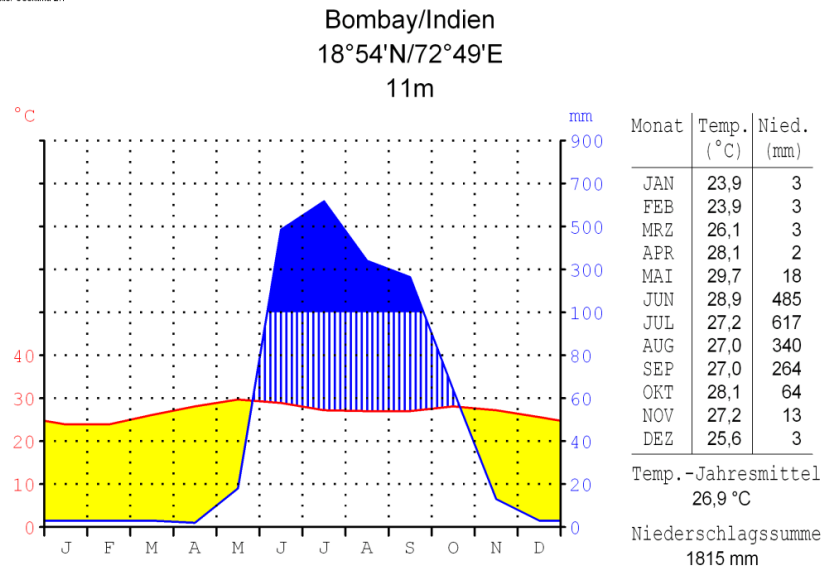


Abbildung 4: Liniendiagramm random Beispiel

### 5.3 Verlaufsansicht (Raumdetails)

- Liniendiagramm für Temperatur, Luftfeuchtigkeit, Luftqualität über wählbaren Zeitraum (Tag / Woche / Monat).
- Grenzwerte als horizontale Referenzlinien eingeblendet.
- Vergangene Grenzwertverletzungen farblich markiert.
- Datenlücken (Datenschutz, Ausfall) klar als graue Bereiche gekennzeichnet.
- Granularität abhängig von Rolle (Mitarbeiter: Rohdaten; Abteilungsleitung: Tagesdurchschnitte für Büros).

### 5.4 Dashboard (Abteilungsleitung)

- Rasteransicht aller Räume der Abteilung mit aktuellen Messwerten und Warnstatus.
- Anzahl aktiver Grenzwertverletzungen pro Raum.
- Verlinkung zu Raumdetailansicht mit reduzierter Granularität für Büros.

### 5.5 Dashboard (Höheres Management)

- Firmenweites Dashboard: Aggregierte Werte nach Abteilungen.
- Balkendiagramm: Anzahl Grenzwertverletzungen pro Abteilung (anonymisiert).
- Trendlinie über Wochen/Monate.

### 5.6 Konfigurationsansicht (Gebäudeadmin)

- Liste aller Räume mit aktuellen Grenzwerten.
- Inline-Bearbeitung der Grenzwerte pro Raum und Messgröße.

- Tipp-Verwaltung: Liste, Erstellen, Bearbeiten, Löschen von Raumklima-Tipps.

## 5.7 Gerätemanagement (Systemadmin)

- Übersicht aller Raspberry Pis und Sensorstationen mit Status (Online/Offline).
- Anlegen neuer Geräte inkl. automatisch generierter ID.
- Zuweisung von Sensorstationen zu Raspberry Pis und Räumen.
- Benutzerverwaltung: Nutzer:innen anlegen, Rollen zuweisen, Räumen zuordnen.

# 6 Projektplan

## 6.1 Verantwortlichkeiten

| Person            | Zuständigkeit                                                                         |
|-------------------|---------------------------------------------------------------------------------------|
| Maria Kuhn        | Arduino-Entwicklung (C/C++, BLE-Protokoll), Frontend (React/TS)                       |
| Fabienne Schedler | Frontend (React/TS, PrimeReact, GUI-Prototyp, Mobile-First)                           |
| Emma Danko        | Raspberry Pi (Python, BLE Central, SQLite), Backend (Spring Boot)                     |
| Prahbdip Singh    | Raspberry Pi (Python, REST-Schnittstellen, Docker), Backend (Spring Boot, PostgreSQL) |
| Adriano Paganini  | Arduino-Entwicklung (C/C++, Display/LED/Buttons), Backend (Spring Boot, API Design)   |
| <b>Alle</b>       | Softwarekonzept, Testing, Dokumentation, Code Review                                  |

## 6.2 Meilensteine und Zeitplan

| Datum      | Meilenstein                             | Inhalt                                                                                                            |
|------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 12.03.2026 | <b>Konzept V1</b>                       | Systemüberblick, Use Cases, Klassendiagramm V1, GUI Prototyp, Projektplan                                         |
| 19.03.2026 | <b>Konzept V2</b>                       | Sequenzdiagramme, API Dokumentation, Ausfallssicherheit, Klassendiagramm V2                                       |
| 27.03.2026 | <b>Konzept Final</b>                    | Überarbeitetes Gesamtkonzept nach Feedback                                                                        |
| KW 14–15   | <b>M1: Hardware Setup</b>               | Arduino + Sensoren funktionsfähig; BLE-Übertragung zum RPi; Raspberry Pi OS + Docker eingerichtet                 |
| KW 16–17   | <b>M2: Backend Grundgerüst</b>          | Spring Boot Skeleton aufgesetzt; PostgreSQL Datenbankschema implementiert; Basis-REST-API (OpenAPI); Login/Rollen |
| KW 18–19   | <b>M3: Datenfluss End-to-End</b>        | Kompletter Datenfluss Arduino → RPi → Backend funktionsfähig; Grenzwertberechnung am RPi; Datenschutzlogik        |
| KW 20–21   | <b>M4: Frontend + Warnungen</b>         | Rollenabhängige Dashboards; Verlaufsdiagramme; Grenzwertwarnungen in Webapp und auf Sensorstation                 |
| KW 22–23   | <b>M5: Ausfallssicherheit + Testing</b> | Ausfallszenarien abgesichert; SonarQube integriert; $\geq 65\%$ Testabdeckung                                     |
| KW 24      | <b>M6: Finalisierung</b>                | Bugfixes; Dokumentation vollständig; finale Abgabe (18.06.2026)                                                   |

## 6.3 Inkrementelle Entwicklung

- Wir entwickeln das System in zwei parallelen Tracks: **Hardware-Track** (Arduino + Raspberry Pi) und **Software-Track** (Backend + Frontend).
- Beide Tracks werden ab M3 integriert.
- Jede Woche wird im Team eine kurze Sync-Session abgehalten, um Fortschritte und Blocker zu besprechen.
- GitLab wird für Issues, Merge Requests und Code Review genutzt. Das Wiki enthält alle Konzept-Dokumente.
- SonarQube wird ab Beginn der Backend-Entwicklung (KW 14) integriert, nicht erst kurz vor der Abgabe.

## 6.4 Zusätzliche Aufgaben

- Zwischenbericht: Einplanen in KW 18 (nach M3).
- Abschlussbericht: Einplanen in KW 23–24.
- Präsentation: Vorbereitung in KW 24.
- Testdaten: Realistische Testdaten für alle Rollen und Szenarien werden in KW 20 erstellt.